

MASTER BUILD SPEC v2 — "ASTRA Platform"

One company. Six products. One shared brain. One unforgettable interface.

You are a senior AI systems architect + full-stack engineer. Build a complete, production-grade, industry-grade AI platform from zero. This is NOT a demo, NOT a prototype — the word "prototype" is banned in this repo. Every module runs real ML models, streams real data in real time, persists state, degrades gracefully, and is measurably benchmarked. Build phase by phase; never advance past a failed gate.

1. WHAT WE ARE BUILDING (understand this before writing code)

ASTRA Platform — a unified autonomous AI operations platform where AI agents perform the jobs of human employees. Six products share one industrial core and one shared memory:

| Layer | Codename | What it is |

|---|---|---|

| Core runtime | `astra-core` | Durable event-sourced agent runtime: every step persisted, runs survive crashes, pause/resume for human approval across days, full replay/time-travel, per-step cost metering, sandboxed tool execution |

| AI Gateway | `sentinel` | LLM security firewall — EVERY model call in the platform passes through it: real-time prompt-injection/jailbreak/PII detection (Aadhaar/PAN/phone for India), token-level streaming output scanning, per-tenant quotas, cost metering |

| Telemetry bus | `pulse` | Real-time logs/metrics/traces/security events → ClickHouse + Isolation Forest anomaly detection. Feeds MEDIC and SHIELD from one bus |

| Context Fabric | `loom` | The shared brain (spec in §3): everything any module learns is stored once, with provenance ("born in SHIELD"), and every other module can read it — so users never redo onboarding when they adopt another module |

| Evolution engine | `forge` | Self-improving skill library: failures become new tools/prompts, validated by an eval harness, promoted only when measurably better. Git is the substrate |

| Model service | `model-forge` | Our own post-trained domain model (text-to-SQL via SFT+GRPO), quantized, served behind `sentinel`, used inside MEDIC |

Product modules:

1. **MEDIC — AI SRE**: on-call engineer agent. `pulse` streams telemetry → anomaly detected → LLM correlates with deploys → queries ClickHouse + code RAG → ranked hypotheses → reproduces in

Docker sandbox → opens GitHub PR → human approves.

2. **OPERATOR — vision computer-use agent**: automation where no API exists. Screenshot → OmniParser v2 → Set-of-Mark → VLM acts via Playwright → screenshot-diff verification → replan → trajectories stored in `forge`/`loom` so repeated tasks skip planning. Live copilot mode.

3. **SHIELD — AI SOC analyst** (defensive only): lab attacks (Atomic Red Team) → `pulse` ingests auth logs/network flows → statistical + LLM correlation → MITRE ATT&CK mapping → Neo4j attack graph → containment recommendation behind approval gate → auto postmortem.

4. **VAANI — voice AI employee**: full-duplex streaming voice agent (Silero VAD → faster-whisper STT → LLM with tenant RAG → Piper TTS, barge-in <100ms, p95 <1.5s) that completes work — books

calendar, writes CRM, sends WhatsApp — through `astra-core`.

5. **FORGE — self-evolving engine**: nightly consolidator mines failures → proposes tools/prompt patches → eval harness + LLM-judge in sandbox → promotion only on measured improvement → Git

commit. Publishes a week-over-week success-rate chart.

6. **MODEL-FORGE — our own model**: QLoRA SFT + GRPO on Qwen2.5-3B/7B, reward = SQL

executes and

returns correct rows. Trained on free Colab/Kaggle GPU, served quantized, registered in `sentinel`.

2. HOW IT IS USED — SOLO MODE & FUSION WORKSPACE (core UX + architecture requirement)

Users must be able to use the platform two ways, and switch freely:

A. SOLO MODE — one module, nothing else.

- A user who only wants cybersecurity lands directly on SHIELD: `python main.py --profile shield` boots ONLY the core services SHIELD needs. No MEDIC, no VAANI running, no resources wasted.
- The console has a module switcher (top-left "waffle"): MEDIC · OPERATOR · SHIELD · VAANI · FORGE.
- Clicking one enters it directly. Modules the user has never opened show as "available — off".
- Going solo is one click and seconds, not setup.

B. FUSION WORKSPACE — everything combined.

- A dedicated unified workspace ("Fusion") where all modules run together and cooperate: a mission-control view with a single cross-module timeline, all live streams side by side, and cross-module workflows — e.g., "SHIELD incident → MEDIC investigates the affected service → OPERATOR verifies the fix on the vendor portal → report compiled."
- Module-as-tool: each module exposes its capabilities as tools in the `astra-core` registry, so one module's agent can invoke another (MEDIC's investigator can call SHIELD's threat check; VAANI can trigger a MEDIC health check). Every cross-module call is logged and provenance-stamped.
- Fusion requires `--profile all`; the launcher prints which modules are up.

****C. THE PROMISE:**** data entered or learned in any module belongs to the USER/TENANT, not to the module. Start solo today, adopt another module next quarter — everything is already there. This is enforced by `loom` (§3), never by copy-paste between modules.

3. LOOM — THE SHARED CONTEXT FABRIC (the anti-re-onboarding system)

****Principle:**** a user must never enter the same information twice, and every module must be aware of what the user did elsewhere — with clear labels of origin.

A. What lives in LOOM (shared across ALL modules, one store):

- ****Org profile**** (filled ONCE): company name, industry, tech stack, services, databases, APIs, team, working hours, compliance needs. Every module reads it — MEDIC knows the tech stack, SHIELD knows the attack surface, VAANI knows the business.
- ****Knowledge base****: any document uploaded anywhere (VAANI tenant docs, MEDIC runbooks, SHIELD policies) → one Qdrant collection with per-module visibility.
- ****Vault****: credentials/secrets saved once, usable by any module the user permits.
- ****Learned artifacts****: anomalies, incidents, hypotheses, patches, trajectories, skills, thresholds, alert rules, notification preferences.
- ****Run history****: every agent run from every module in one queryable timeline.

B. Provenance is mandatory on EVERY shared item. Schema:

`context\_items(id, tenant\_id, origin\_module, origin\_run\_id, kind, title, summary, payload, created\_at, shared\_with\_modules[], used\_by[])` — plus human-readable provenance strings. Every item carries: where it was born ("origin: SHIELD, run #482, 2026-09-05"), and where it has been used ("used_by: MEDIC investigator 2026-09-07").

C. Concrete cross-module flows that MUST work (implement exactly these):

1. User uses ONLY SHIELD for a month → SHIELD records incident INC-42 ("brute force on host db-1").

User later enables MEDIC → MEDIC's console immediately shows a context card: "From your SHIELD

work: recent security incident on db-1 — relevance to error spikes: high" — no re-onboarding, no re-entering infra details (org profile already there).

2. User uploads pricing PDFs for VAANI → MEDIC's investigator and SHIELD's analyst can retrieve the same docs from the shared KB, each labeled "source: VAANI knowledge base".

3. SHIELD finds a vulnerability → MEDIC's next investigation receives it as context; MEDIC's PR description cites "related SHIELD incident INC-42" with a link.

4. OPERATOR learns a portal-login trajectory → VAANI's booking flow that needs the same portal reuses it via FORGE, stamped "trajectory learned by OPERATOR".

5. FORGE skills learned from MEDIC failures are offered to OPERATOR and SHIELD agents.

****D. Consent & control (the sharing matrix):****

- Settings page: per-module toggles for what each module may READ from LOOM (org profile / docs / vault / incidents / skills), default = read context, never read vault without explicit grant.
- Every module's console has a "Context from your other workspaces" panel: cards with origin stamps, relevance, and a "stop sharing this with me" control.
- Provenance is shown in the UI with stamp badges (see design system): `FROM SHIELD`, `USED BY MEDIC`.

****E. Architecture:**** `loom` is a FastAPI service + Postgres tables + Qdrant namespace; modules integrate via a thin Python client (`loom.client`) and never write cross-module state anywhere else. LOOM never duplicates the origin module's raw data — it stores references + summaries + payloads.

4. NON-NEGOTIABLE ENGINEERING PRINCIPLES

1. ****NOTHING HARDCODED.**** No fixed inputs/outputs. Thresholds, prompts, rules, reference ranges,

attack signatures, alert rules live in the DB, editable in the console. Anomaly thresholds are learned from live baselines. Alert rules are parsed by an LLM from plain English into evaluated conditions. Agents work on data/sources they've never seen (new log schema, new website, new docs).

2. ****REAL-TIME EVERYWHERE.**** Every dashboard streams via WebSocket/SSE. LLM tokens stream. STT/TTS

stream. `sentinel` scans the response stream chunk-by-chunk. Voice is full-duplex. Live latency displayed on every surface.

3. ****DURABILITY.**** Event-sourced runs in Postgres; `kill -9` mid-run → resume exactly; approval gates survive restarts; full replay from any event.

4. ****PROVENANCE.**** Any artifact an agent produces that enters LOOM carries origin + usage stamps.

Untracked data is a bug.

5. ****PRODUCTION GRADE means:**** graceful degradation on every external dependency with defined

fallback paths; retries with backoff; circuit breakers; per-tenant rate limits; secrets only via env/vault; structured JSON logs; OpenTelemetry tracing; health/readiness endpoints; Alembic migrations; >70% test coverage on `astra-core`, `sentinel`, `loom`; load-tested; README runbook.

6. ****EVERY MODULE HAS A BENCHMARK.**** MEDIC: MTBD/MTTR on injected faults. OPERATOR: success rate on

a 20-task web suite. SHIELD: detection rate + false-positive rate. VAANI: p95 latency + task completion. FORGE: week-over-week eval score. sentinel: detection accuracy + added latency. Stored over time, charted in the console. No benchmark = not done.

5. TECH STACK (pinned, free/open-source)

Python 3.11+, FastAPI, Uvicorn, Pydantic v2, asyncio, SQLAlchemy 2 + Alembic, Postgres 16, Redis 7
.

LangGraph on top of the astra-core event store · scikit-learn (Isolation Forest), sentence-transformers, ONNX Runtime (deberta-v3 prompt-injection), faster-whisper, Silero VAD, Piper TTS, torch/TRL/PEFT (training only), vLLM or llama.cpp · OmniParser v2 + Set-of-Mark, Playwright, OpenCV · OpenTelemetry, ClickHouse, Prometheus, Grafana, Loki · Neo4j community · Next.js 15 (App Router), TypeScript, Tailwind, Recharts, react-flow (theming per §8 — do NOT use stock shadcn look) · Docker Compose profiles (core, sre, soc, voice, operator, observability, all), MinIO, Qdrant, MailHog, GitHub Actions.

6. MONOREPO STRUCTURE

```
...
astra/
├── main.py          # ONE command launcher (§7)
├── docker-compose.yml # profiles: core, sre, soc, voice, operator, observability, all
├── .env.example
├── backend/app/
│   ├── core/      # event store, run engine, approvals, sandbox, tool registry, replay
│   ├── sentinel/  # gateway proxy, detection layers, stream scanner, quotas
│   ├── pulse/     # OTEL ingestion, ClickHouse queries, anomaly jobs
│   ├── loom/      # CONTEXT FABRIC: context_items, provenance, sharing matrix, KB, org profile
│   ├── medic/     # detector, investigator, reproducer, fixer
│   ├── operator/  # parser, SoM, executor, verifier, trajectory memory
│   ├── shield/    # correlation, ATT&CK mapping, Neo4j graph, postmortems
│   ├── vaani/     # WS voice gateway, VAD, STT/TTS streams, barge-in, call state
│   ├── forge/     # skill library, eval harness, consolidator
│   ├── model_forge/ # training notebooks (separate), serving adapter, model registry
│   ├── shared/    # auth(JWT), tenancy, DB, vault, config, streaming utils, loom client, LLM client
│   └── api/       # routers per module + /ws endpoints
├── alembic/
├── backend/tests/
├── frontend/      # Next.js console (§8)
└── docs/          # runbook per module, benchmarks, architecture, loom data-flow docs
...
```

7. `main.py` — ONE-COMMAND LAUNCHER (required)

```
`python main.py [--profile all|core|sre|soc|voice|operator|shield] [--no-frontend] [--rebuild]`:
```

1. Validate/create `.env`; report exactly which free keys are missing and where to get them.
2. Start the Docker Compose profile for the chosen mode; wait for each healthcheck. Solo mode boots

only that module's dependencies (e.g., shield: core+loom+pulse+neo4j, no voice/operator services).

3. Run Alembic migrations; seed ONLY reference/config data — never fake business data.

4. Start FastAPI backend (port 8000) and Next.js frontend (port 3000) as managed subprocesses with

```
`[api]`/`[web]` log prefixes; poll health endpoints; print a live status table (which modules up, which off); on crash show the failing component's last 50 log lines.
```

5. Ctrl-C → graceful shutdown in reverse order.

A clean clone + `python main.py` must reach a fully working system on macOS (Apple Silicon) and Linux.

8. FRONTEND — THE "BLUEPRINT" DESIGN LANGUAGE (not another generic AI dashboard)

****Hard ban list:**** dark background with neon green/purple glow · purple-blue gradient heroes · glassmorphism blur cards · default unthemed shadow look · generic SaaS sidebar with rounded purple active states · floating gradient orbs · everything-in-dark-mode-as-default. If it looks like a typical AI startup template, redo it.

****The direction — "engineer's field notebook meets mission-control printout":****

- ****Base:**** warm paper off-white `#FAF7F2`, ink `#141414`, hairline rules and a faint graph-paper grid on canvas areas (like technical drawing sheets). Dark variant is "control room": desaturated graphite `#1C1E21` with warm off-white text — never pure black, never neon.
- ****Accent:**** one signal color — safety orange `#E85D2A` — used sparingly for live/active states and alerts. Status palette is muted: amber `#C7952C`, olive `#6E7F3E`, oxide red `#B3402E`, slate blue `#4A6274`. No saturated rainbow UI.
- ****Type:**** display headings in a characterful grotesk or serif (e.g., Space Grotesk or Fraunces); all technical labels, timestamps, IDs, and data in a monospace (JetBrains Mono / IBM Plex Mono), uppercase, letter-spaced — like instrument labels. Body text is highly readable at density.
- ****Signature components:****
 - ****Provenance stamps:**** small rubber-stamp-style badges (1px ink border, slight rotation $\pm 2^\circ$, uppercase mono) — `FROM SHIELD`, `USED BY MEDIC`, `SHARED`, `LOCAL ONLY`. Every loom item shows them.
 - ****Corner ticks / crosshair markers**** on panels (like blueprint drawings) instead of rounded shadows.
 - ****Dotted connectors**** for graphs/timelines; execution trees and Neo4j attack graphs rendered with thin ink lines, orange for the live/active path.
 - ****Dense instrument panels:**** metrics as compact mono readouts with tiny sparklines, like a flight deck, not marketing cards.
- ****Layout:**** a top command bar (workspace switcher: SOLO modules Δ + FUSION, global search, run status, cost meter) rather than a fat sidebar; module pages are dense workbenches with a right rail showing "Context from your other workspaces" (loom cards with origin stamps).
- ****Motion:**** precise and minimal (150–250ms ease-out); no glow, no pulse, no parallax.
- ****Frontend surfaces (all live via SSE/WS):**** Overview (cross-module timeline, costs, runs) · Runs (react-flow execution trees, event timeline, approve/resume/replay, time-travel) · Sentinel (live traffic, flagged requests, block rate, latency overhead) · MEDIC (live anomaly stream, incident war-room, hypothesis cards, PR links) · OPERATOR (live screen + SoM overlay, action log, task leaderboard) · SHIELD (attack graph, incident narratives, ATT&CK mapping) · VAANI (call console: live transcript, audio meters, latency panel, actions taken) · FORGE (eval leaderboard, improvement chart) · LOOM (context explorer: everything the platform knows about you, filterable by origin module, with provenance stamps) · Settings (org profile, sharing matrix, keys vault, rules editors for every threshold/alert/detection rule — the anti-hardcoding surface).

9. MODELS & API KEYS — FREE TIERS ONLY (document in .env.example with links)

| Env var | Provider | Get it | Free-tier use |

|---|---|---|---|

| `GEMINI_API_KEY` | Google AI Studio | aistudio.google.com | Gemini 2.5 Flash (streaming, tool calls, VISION for OPERATOR), Flash-Lite for cheap paths, embeddings |

| `GROQ_API_KEY` | Groq | console.groq.com | Llama 3.3 70B/8B instant streaming; Whisper large-v3 STT for VAANI |

| `HF_TOKEN` | HuggingFace | huggingface.co | ONNX prompt-injection + PII models, OmniParser weights, datasets |

| `*(no key)*` | Ollama local | ollama.com | qwen2.5vl:7b (vision fallback), qwen2.5-coder:7b, llama3.2:3b, nomic-embed-text — every cloud model MUST have a local fallback route |

| *(no key)* | local | — | faster-whisper, Piper TTS, Silero VAD, scikit-learn, ONNX Runtime, sentence-transformers |
| *(dev only)* | Deepgram credit / ElevenLabs free chars / Sarvam dev credits | their consoles |
Optional upgrades; never required |
| *(training)* | Google Colab free T4 / Kaggle 30 GPU-hrs/wk | colab.research.google.com | MODEL-FORGE SFT + GRPO |

Every LLM-touching component implements the fallback chain: free cloud → Groq → local Ollama → defined degradation (never a crash, never a canned hardcoded answer).

10. KEY TECHNICAL SPECS (condensed — implement fully)

- ****astra-core:**** tables `runs`, `run\_events` (append-only, full payloads + token/₹ cost), `approvals`, `tools`, `tool\_executions`, `schedules`. LangGraph nodes write events; crash recovery from last checkpoint; `AWAITING\_APPROVAL` persists across restarts; APIs: POST /runs, GET /runs/{id}/events?since=, POST /runs/{id}/approve, GET /runs/{id}/replay/{n}. Every tool execution runs in a resource-limited Docker sandbox, output streamed over WebSocket.
- ****sentinel:**** OpenAI-compatible proxy `/v1/chat/completions` (+ Gemini passthrough); input pipeline
 <120ms p95: Indic-PII/jailbreak heuristics → ONNX deberta-v3 injection classifier (CPU) → embedding
 similarity vs attack-pattern DB (Qdrant) → LLM-as-judge only for borderline (async). Output: chunk-level
 stream scanning with mid-stream redaction/stop. Rules are DB rows, editable per tenant.
- ****pulse + MEDIC:**** demo microservices (checkout/payments/inventory) instrumented with OTel → Collector → ClickHouse; random (unscripted) fault injector; Isolation Forest with learned baselines; investigator uses ClickHouse SQL (via MODEL-FORGE model when available) + code RAG + deploy correlation; reproducer runs hypotheses in sandbox; fixer opens GitHub PRs; run pauses at approval.
- ****SHIELD:**** Wazuh/auditd + Suricata in a lab; events through pulse; LangGraph correlation → MITRE ATT&CK mapping (DB-backed) → Neo4j attack graph → containment behind approval → postmortem.
- ****OPERATOR:**** OmniParser v2 → SoM overlay → VLM (Gemini Flash; Ollama qwen2.5vl local fallback) →
 Playwright execution → screenshot-diff verification → replan (bounded retries → human gate); trajectories embedded into loom/forgo for reuse.
- ****VAANI:**** browser mic → WS audio (16kHz) (telephony adapter interface ready for Exotel/Twilio later);
 Silero VAD → streaming faster-whisper (Groq Whisper as cloud option) → LLM + tenant RAG (docs via loom) → Piper TTS streamed sentence-level; barge-in cancels TTS <100ms; enforced latency budget
 (VAD 30ms / STT 250ms / LLM first token 300ms / TTS 250ms → p95 <1.5s) displayed live; tool calls
 (calendar/CRM/WhatsApp) via astra-core with approval + replay.
- ****FORGE:**** eval harness (programmatic checks + LLM-judge) in sandbox; nightly consolidator mines
 `run\_events` failures → candidate tool/prompt patch → evaluated vs current → promoted only on measured improvement → Git commit; weekly success-rate chart.
- ****MODEL-FORGE:**** seed data from a free frontier model → QLoRA SFT (TRL+PEFT, Qwen2.5-3B/7B) →
 GRPO with executable-SQL reward → quantized GGUF → served → registered in sentinel → wired into
 MEDIC. Colab/Kaggle free GPU notebooks. TensorBoard tracking.

11. BUILD ORDER — PHASE GATES (never advance past a failed gate)

- **Phase 0 — Skeleton:** repo, compose profiles, Alembic, JWT auth + tenancy, `main.py` launcher, Next.js shell in the Blueprint design language (command bar, module switcher, SOLO/FUSION entry).
GATE: clean clone → `python main.py` → healthy stack, login, switcher works, theme applied.
- **Phase 1 — Core + Sentinel + LOOM:** event-sourced run engine, approvals, sandbox, tool registry;
sentinel with input/output/stream scanning; loom with `context\_items`, provenance, org profile, sharing matrix, KB. GATE: run survives `kill -9` and resumes; every LLM call passes sentinel (<120ms overhead, red-team set blocked, PII redacted mid-stream); an artifact created by one seeded module appears in another module's context panel with correct FROM/USED-BY stamps.
- **Phase 2 — Pulse + MEDIC:** GATE: 10 random injected faults → detection, correct service in top-3 hypotheses ≥7/10, PR opened, MTTD <2min, approval in console, incident written to loom with provenance and visible to SHIELD's context panel.
- **Phase 3 — OPERATOR:** GATE: ≥50% success on 20 unfamiliar-site tasks, zero host access from sandbox, ≥3 demonstrated self-corrections, trajectories in loom reused by a repeat task.
- **Phase 4 — SHIELD:** GATE: 5/5 lab attacks detected with correct ATT&CK mapping, FP rate <10%
on 24h benign traffic, incident graphs + postmortems, provenance into loom, MEDIC sees it.
- **Phase 5 — VAANI:** GATE: p95 <1.5s, barge-in <100ms, real booking end-to-end via astra-core, 10/10 conversations with zero hardcoded replies, transcripts+bookings into loom (labeled FROM VAANI).
- **Phase 6 — FORGE + MODEL-FORGE:** GATE: eval chart improving over ≥3 consecutive weekly runs;
our model beats its base on a held-out SQL test set; registered in sentinel and used by MEDIC.
- **Phase 7 — Hardening:** load test (200 concurrent WS streams + 50 runs), chaos testing, security review, runbooks, CI. GATE: no data loss/deadlock under chaos; green CI; runbook complete; solo-mode boot of each module verified independently.

12. WHAT NOT TO DO

No mocked data in production paths · no canned responses standing in for ML · no
`if input == X: return Y` anywhere · no unscripted-looking demo scripts masquerading as agents ·
no secrets in code · no module done without its benchmark chart · no artifact entering loom without
provenance · no cloud-only dependency without local fallback · no generic AI/neon/glassmorphism
UI ·
no module that requires re-entering data another module already has.

Begin with Phase 0. After each phase report: what was built, how to run it, gate results with real numbers, and what's next. If a gate fails, fix it before moving on.